

Lecture 2: Lambda Expressions and Lists

COMP2221: Functional programming

Maximilien Gadouleau

`m.r.gadouleau@durham.ac.uk`

λ -calculus

- λ -calculus: rule system to describe computations solely via function abstraction and application
 - Inspired functional programming languages
 - Simplest known Turing-complete programming language
- Proof of Turing-completeness:
- `https://turingarchive.kings.cam.ac.uk/
publications-lectures-and-talks-amtb/amt-b-11`

Definition of λ -expressions

- Inductive definition to build all λ -expressions:
 - Variable v is a λ -expression.
 - Variables represented by lower-case letters
 - If M is a λ -expression, then $(\lambda v.M)$ is a λ -expression.
 - Abstraction aka definition of a function with parameter v and body M
 - λ denotes start of function definition
 - If M and N are λ -expressions, then (MN) is a λ -expression.
 - Application of M to N
- Note: Functions take exactly one argument, *currying* used to model multi-argument functions

Valid λ -expressions:

- $x \rightarrow$ a variable
- $(\lambda x.x) \rightarrow$ the identity function
- $((\lambda x.x)e) \rightarrow$ the identity function applied to an expression e
- $(\lambda x.(\lambda y.(xy))) \rightarrow$ nested function, i.e. currying
- $((((\lambda x.(\lambda y.(xy)))a)b) \rightarrow$ nested function applied to expressions a and b

Conventions to avoid ambiguity

- Application is left-associative, e.g., MNP is equivalent to $((M\ N)\ P)$; instead of $(M\ (N\ P))$
- Abstraction is right-associative
- Application has precedence over abstraction, e.g. $\lambda x.\lambda y.xy$ is equivalent to $\lambda x.(\lambda y.(xy))$; instead of $\lambda x.(\lambda y.x)y$

Free vs. bound variables

Bound variables

A variable is *bound* if it occurs in a function that takes a variable of the same name as input. For instance, x in $\lambda x.x$ is bound. A variable binds to the closest function argument considering its enclosing functions.

Free variables

A variable is *free* if it is not bound. For instance, x in $\lambda y.x$ is free.

β -Reduction to evaluate expressions

β -reduction allows to substitute the argument of an abstraction with the value of an application $((\lambda x.M[x])N) \rightarrow (M[x := N])$.

Example 1:

$$(\lambda x.x)a = a$$

$$(\lambda x.y)a = y$$

Example 2:

$$((\lambda x.(\lambda y.(xy)))a)b =$$

$$(\lambda y.(ay))b =$$

$$ab$$

- Normalform: A λ expression is in β -normalform if no β -reduction is possible, i.e., if the expression cannot be reduced any further.

Transformation rules: α -Conversion

α -Conversion to rename variables

α -conversion allows to resolve name conflicts by renaming bound variables as such $(\lambda x.M[x]) \rightarrow (\lambda y.M[y])$.

$$\lambda x.x \equiv \lambda y.y$$

$$\lambda x.(\lambda x.x) \equiv \lambda x.(\lambda y.y)$$

- Prevent capturing free variables when β -reducing expressions
- For instance, β -reduction would change the semantics of the inner function in $(\lambda x.(\lambda y.xy))y$ without prior α -conversion

Lambda expressions

Nameless functions

- As well as giving functions names, we can also construct them *without* names using *lambda expressions*

```
-- The nameless function that takes  
-- a number x and returns x + x  
 $\lambda$ x -> x + x
```

- Use of λ for nameless functions comes from *lambda calculus*, which is a theory of functions.
- It is also a way of formalising the idea of *lazy evaluation* (on which more later)

Use cases for unnamed functions I

- Formalises idea of functions defined using currying

```
add x y = x + y
-- Equivalently
add = \x -> (\y -> x + y)
```

- The latter form emphasises the idea that `add` is a function of one variable that returns a function
- Also useful when returning a function as a result

```
const :: a -> b -> a
const x _ = x
-- Or, perhaps more naturally
const x = \_ -> x
```

“`const` eats an `a` and returns a function which eats a `b` and always returns the same `a`.”

Use cases for unnamed functions II

- What good is a function which always returns the same value?
- Often when using *higher-order* functions, we need a base case that always returns the same value.

```
length' :: [a] -> Int
```

```
length' xs = sum (map (const 1) xs)
```

“The length of a list can be obtained by summing the result of calling `const 1` on every item in the list”

- We will see some more of this when we look at *higher order* functions.

Use cases for unnamed functions III

- Also useful where the function is only used once

```
-- Generate the first n positive odd numbers
odds :: Int -> [Int]
odds n = map f [0..n-1]
  where
    f x = x*2 + 1
```

- Can be simplified (removing the `where` clause)

```
odds :: Int -> [Int]
odds n = map (\x -> x*2 + 1) [0..n-1]
```

Translating between the two forms

- It is always possible to translate between named functions and arguments, and the approach using λ expressions of one argument
- Just move the arguments to the right hand side and put it inside a λ , repeat with remainder until you're done.

```
f a b c = ...  
-- Move formal arguments to right hand side with a lambda  
f = \a b c -> ...  
-- move remaining arguments into new lambdas  
f = \a -> (\b -> (\c -> ...))
```

- Which option fits more naturally is often a style choice
- *Pattern matching* is supported in the argument list in exactly the same way as normal functions

```
head = \(x:_) -> x
```

- I sometimes find it easier to think about *composing* functions or currying by explicitly writing λ expressions

Lists: patterns matching

Representation of lists

- Every non-empty list is created by repeated use of the `(:)` operator “*construct*” that adds an element to the start of a list

`[1, 2, 3, 4] = 1 : (2 : (3 : (4 : [])))`

- This is a representation of a *linked list*
 - Operations on lists such as indexing, or computing the length must therefore *traverse* the list.
- ⇒ Operations such `reverse`, `length`, `(!!)` are linear in the length of the list.
- Getting the `head` and `tail` is constant time, as is `(:)` itself.

Pattern matching on lists

- lists can be used for pattern matching in function definitions

```
startsWithA :: [Char] -> Bool
startsWithA ['a', _, _] = True
startsWithA _ = False
```

- Matches 3-element lists and checks if the first entry is the character 'a'.

Careful

Use patterns in the equations defining a function. Not in the type of the function.

Pattern matches in the equations don't change the *type* of the function. They just say how it should act on particular expressions.

Pattern matching on lists

- How match 'a' and not care how long the list is?
- Can't use literal list syntax. Instead, use list constructor syntax for matching.

```
startsWithA :: [Char] -> Bool
startsWithA ('a':_) = True
startsWithA _      = False
```

- ('a':_) matches any list of length *at least* 1 whose first entry is 'a'.
- The *wildcard* match _ matches anything else.
- This works to match multiple entries too:

```
startsWithAB :: [Char] -> Bool
startsWithAB ('a':'b':_) = True
startsWithAB _          = False
```

Binding variables in pattern matching

- As well as matching literal values, we can also match a (list) pattern, and bind the values.

```
sumTwo :: Num a => [a] -> a
sumTwo (x:y:_) = x + y
```

- Match lists of length *at least* two and sum their first two entries

Example

```
sumTwo [1, 2, 3, 4]
-- introduces the bindings
x = 1
y = 2
_ = [3, 4]
```

- Reminder: can't repeat variable names in bindings (exception `_`)

```
-- Not allowed
sumThree (a:a:b:_) = a + a + b
-- Allowed
second (_,a:_) = a
```

What types of pattern can I match on?

- Patterns are constructed in the same way that we would construct the arguments to the function

```
(&&) :: Bool -> Bool -> Bool
True && True = True
False && _ = False
-- Used as:
a && b
head :: [a] -> a
head (x:_) = x
-- Used as:
head [1, 2, 3] == head (1:[2, 3])
```

- This is a general rule in constructing pattern matches “If I were to call the function, what structure do I want to match?”
- Caveat: can only match “data constructors”

```
-- Not allowed
last :: [a] -> a
last (xs ++ [x]) = x
```

Lists: comprehensions

List comprehensions I: syntax

- In maths, we often use *comprehensions* to construct new *sets* from old ones

$$\{2, 4\} = \{x \mid x \in \{1..5\}, x \bmod 2 = 0\}$$

“The set of all integers x between 1 and 5 such that x is even.”

- Haskell supports similar notation for constructing lists.

```
Prelude> [x | x <- [1..5], x `mod` 2 == 0]  
[2, 4]
```

“The list of all integers x where x is drawn from $[1..5]$ and x is even”

- `x <- [1..5]` is called a *generator*
- Compare Python comprehensions

```
[x for x in range(1, 6) if (x % 2) == 0]
```

List comprehensions II: generators

- Comprehensions can contain multiple generators, separated by commas

```
Prelude> [(x, y) | x <- [1,2,3], y <- [4, 5]]  
[(1,4),(1,5),(2,4),(2,5),(3,4),(3,5)]
```

- Variables in the later generator change faster: analogous to nested loops

```
l = []  
for x in [1, 2, 3]:  
    for y in [4, 5]:  
        l.append((x, y))
```

```
# analogously  
[(x, y) for x in [1, 2, 3] for y in [4, 5]]
```

- Later generators can reference variables from earlier generators

```
Prelude> [(x, y) | x <- [1..3], y <- [x..3]]  
[(1,1),(1,2),(1,3),(2,2),(2,3),(3,3)]
```

“All pairs (x, y) such that $x, y \in \{1, 2, 3\}$ and $y \geq x$ ”

List comprehensions III: guards

- As well as binding variables to values with generators, we can restrict the values using *guards*
- A guard can be any function that returns a `Bool`
- Guards and generators can be freely interspersed, but guards can only refer to variables to their left

```
Prelude> [(x, y) | x <- [1..3], even x, y <- [x..3]]  
[(2, 2), (2, 3)]
```

```
Prelude> [(x, y) | x <- [1..3], y <- [x..3], even x, even y]  
[(2, 2)]
```

```
Prelude> [(x, y) | x <- [1..3], even x, even y, y <- [x..3]]  
error: Variable not in scope: y :: Integer
```

Some examples

- Produce a list of all factors of some positive integer

```
factors :: Int -> [Int]
factors n = [x | x <- [1..n], n `mod` x == 0]
```

- For example

```
> factors 10
[1, 2, 5, 10]
```

- Now we can determine if a number is prime

```
prime :: Int -> Bool
prime n = factors n == [1, n]
```

- And use it to (very expensively) enumerate primes below a limit

```
primes :: Int -> [Int]
primes n = [x | x <- [2..n], prime x]
```